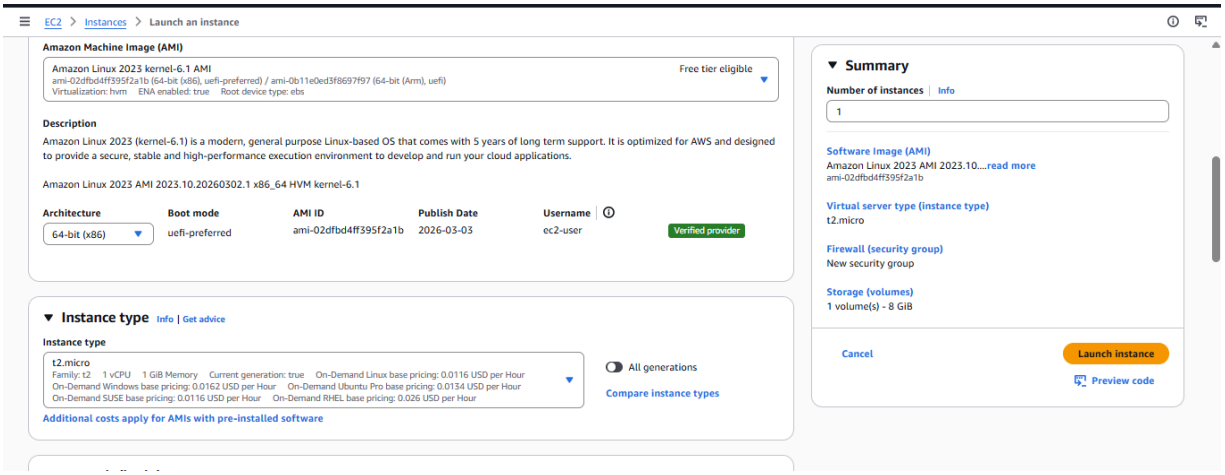
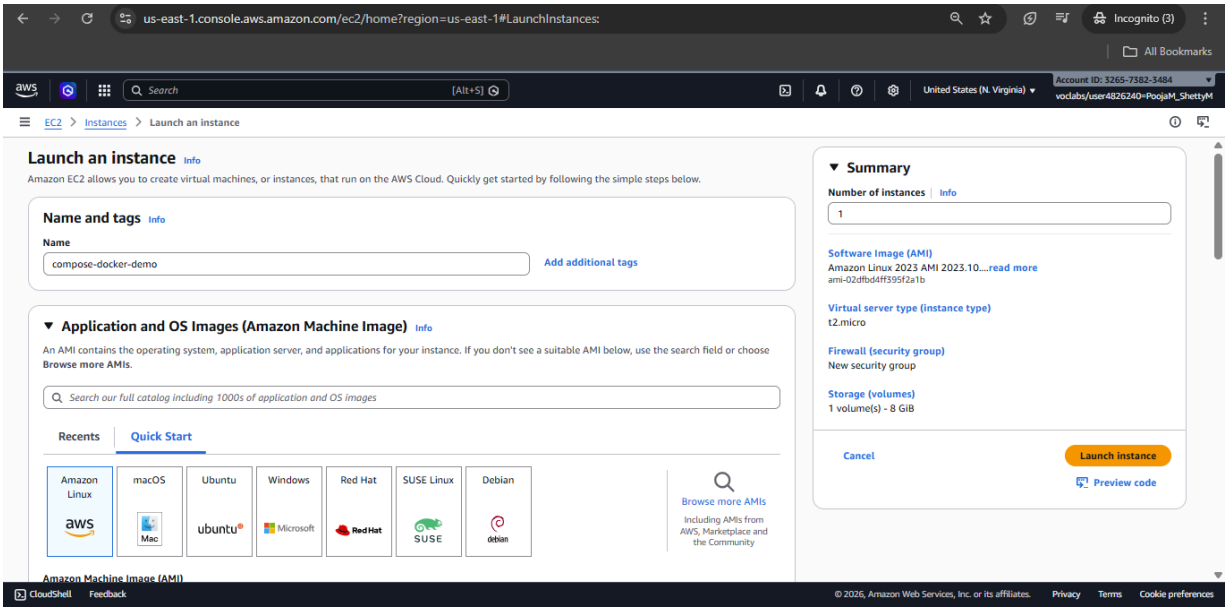
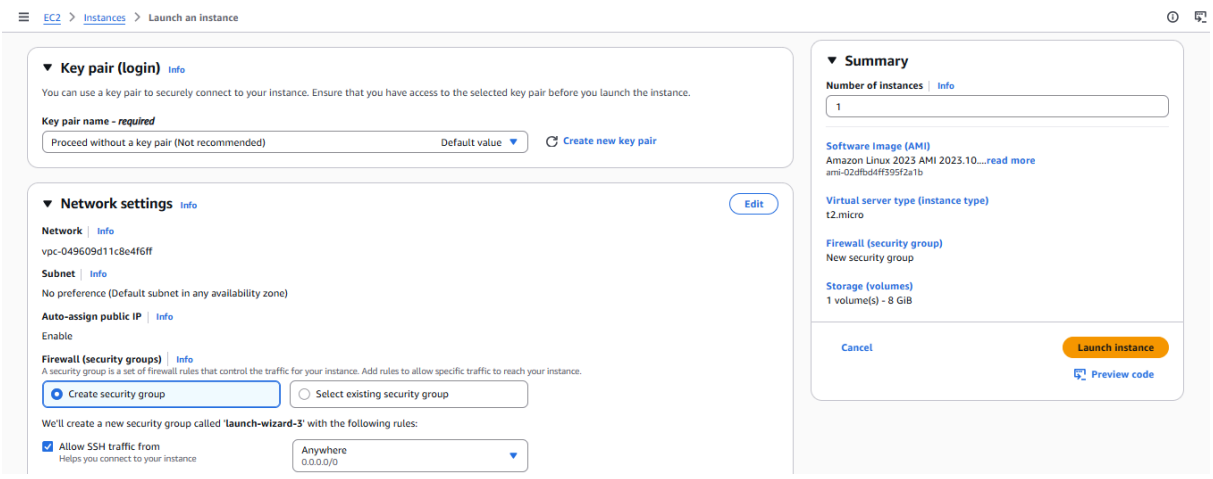
PHASE 1 — AWS SETUP (EC2 + Docker)/ VM

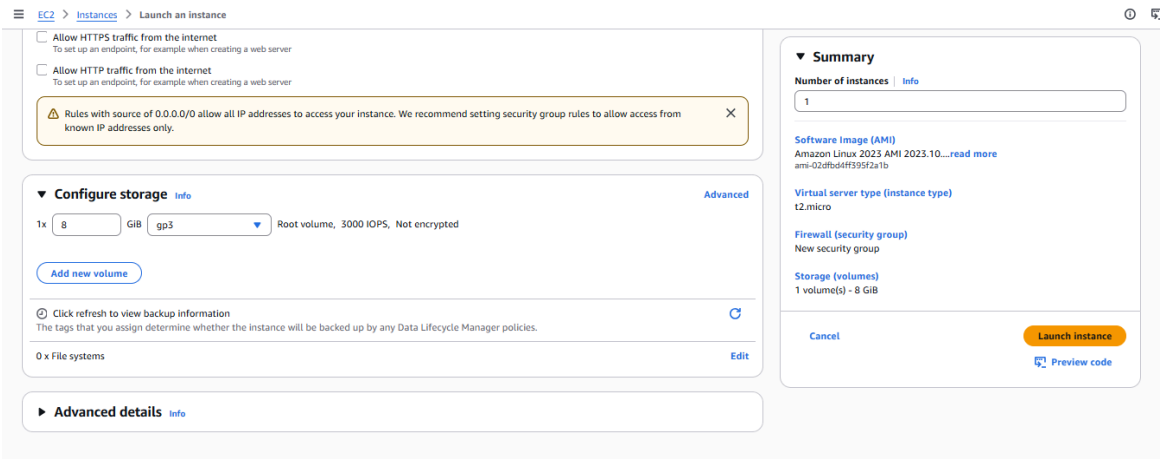
Step 1: Launch EC2 Instance

1. Go to **AWS Learner Lab → Start Lab**
2. Open **EC2 → Launch Instance**
3. Choose:
    - Amazon Linux 2
    - t2.micro

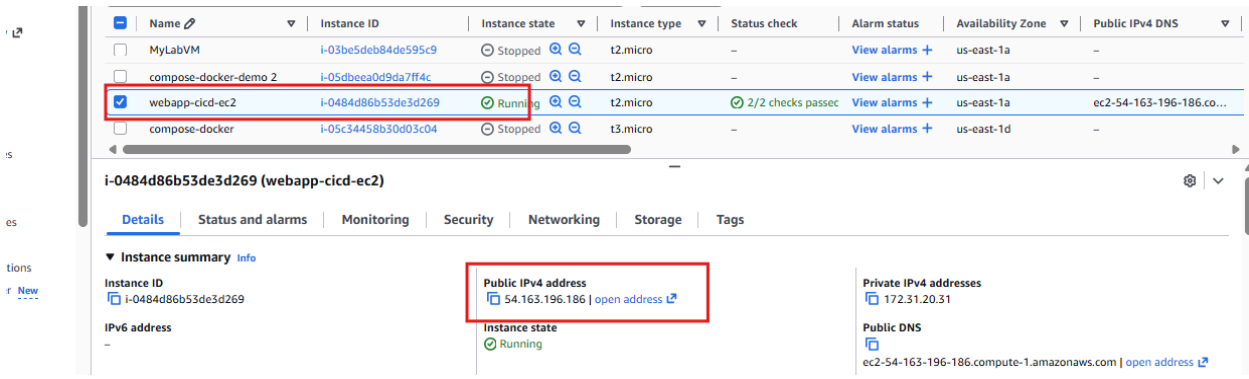




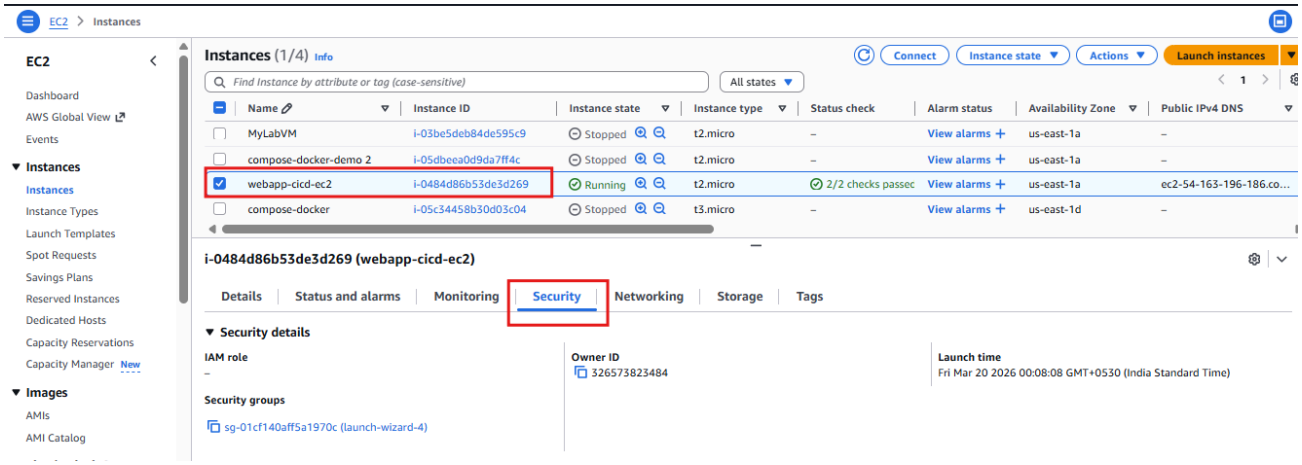


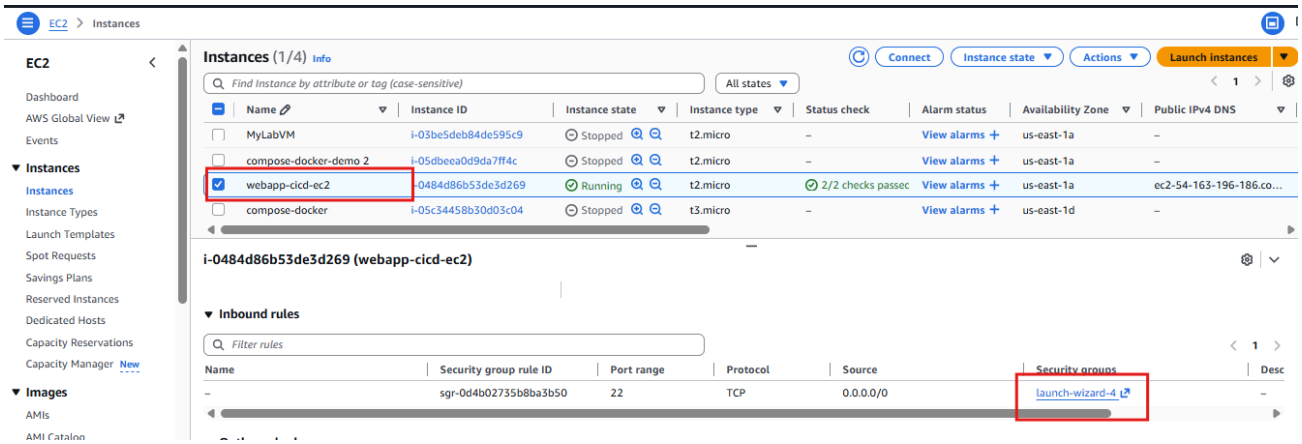


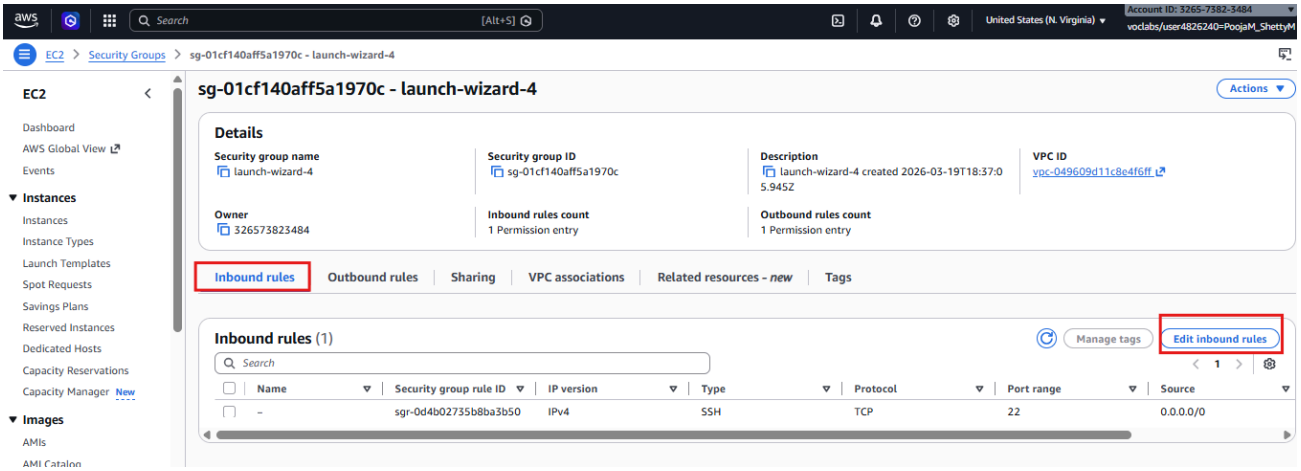
Instance created

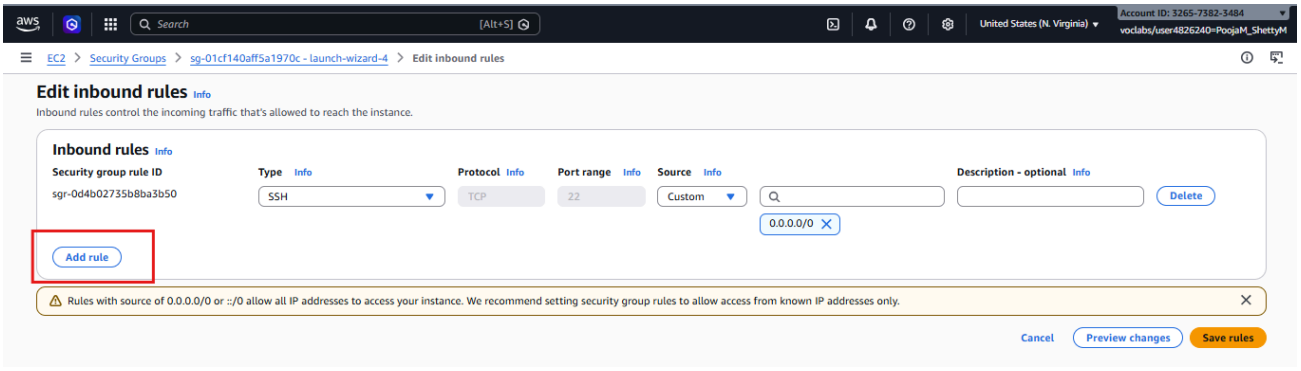


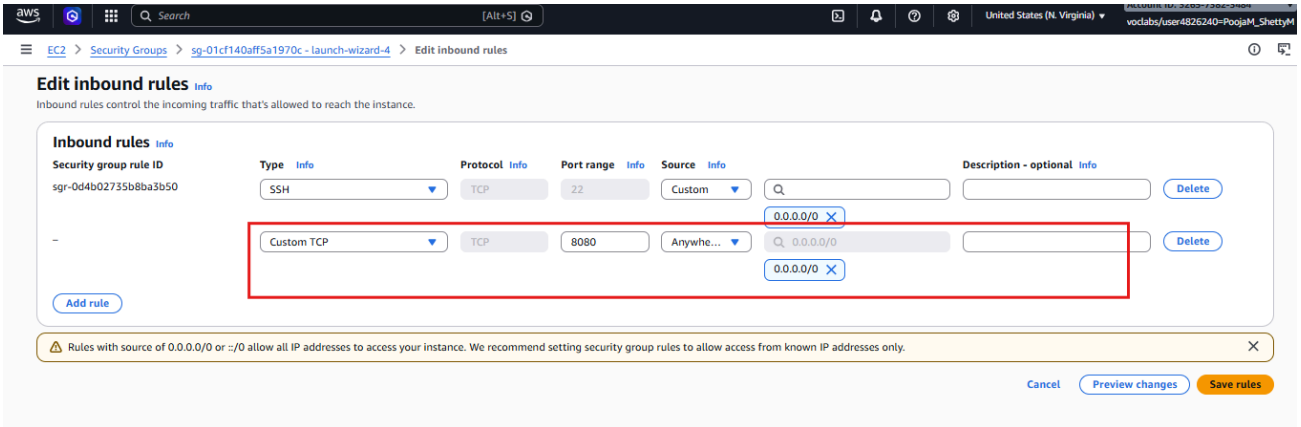
4. Configure Security Group:
   - SSH (22)
   - Custom TCP (8080)

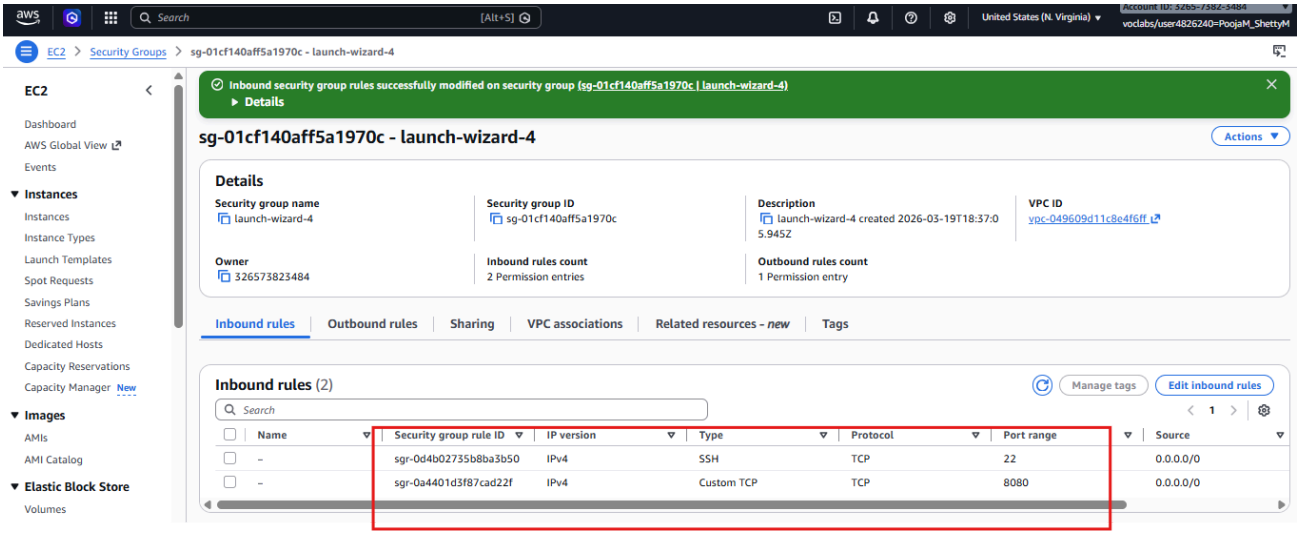





EC2 > Security Groups > sg-01cf140aff5a1970c - launch-wizard-4
sg-01cf140aff5a1970c - launch-wizard-4
Actions
Details
Security group name
launch-wizard-4
Security group ID
sg-01cf140aff5a1970c
Description
launch-wizard-4 created 2026-03-19T18:37:0 5.945Z
VPC ID
vpc-049609d11c8e4f6ff
Owner
326573823484
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (1)
Manage tags
Edit inbound rules
Name
Security group rule ID
IP version
Type
Protocol
Port range
Source
sgr-0d4b02735b8ba3b50
IPv4
SSH
TCP
22
0.0.0.0/0


EC2 > Security Groups > sg-01cf140aff5a1970c - launch-wizard-4 > Edit inbound rules
Edit inbound rules
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional
sgr-0d4b02735b8ba3b50
SSH
TCP
22
Custom
0.0.0.0/0
Delete
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.
Cancel
Preview changes
Save rules

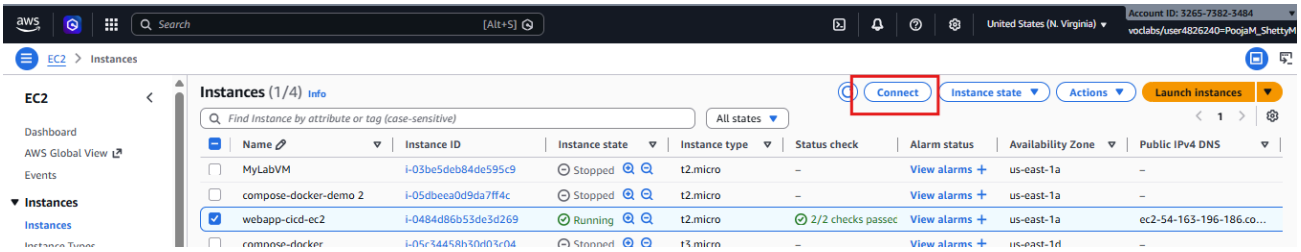

EC2 > Security Groups > sg-01cf140aff5a1970c - launch-wizard-4 > Edit inbound rules
Edit inbound rules
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional
sgr-0d4b02735b8ba3b50
SSH
TCP
22
Custom
0.0.0.0/0
Delete
Custom TCP
TCP
8080
Anywhe...
0.0.0.0/0
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.
Cancel
Preview changes
Save rules


EC2 > Security Groups > sg-01cf140aff5a1970c - launch-wizard-4
Inbound security group rules successfully modified on security group (sg-01cf140aff5a1970c | launch-wizard-4)
Details
sg-01cf140aff5a1970c - launch-wizard-4
Actions
Details
Security group name
launch-wizard-4
Security group ID
sg-01cf140aff5a1970c
Description
launch-wizard-4 created 2026-03-19T18:37:0 5.945Z
VPC ID
vpc-049609d11c8e4f6ff
Owner
326573823484
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Name
Security group rule ID
IP version
Type
Protocol
Port range
Source
sgr-0d4b02735b8ba3b50
IPv4
SSH
TCP
22
0.0.0.0/0
sgr-0a4401d3f87cad22f
IPv4
Custom TCP
TCP
8080
0.0.0.0/0

○

Step 2: Connect to EC2

Use **EC2 Instance Connect**



Step 3: Install Docker

sudo yum update -y

```
[ec2-user@ip-172-31-20-31 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-20-31 ~]$
```

sudo yum install docker -y

```
[ec2-user@ip-172-31-20-31 ~]$ sudo yum install docker -y
Last metadata expiration check: 0:00:30 ago on Thu Mar 19 19:07:48 2026.
Dependencies resolved.
================================================================================================================================
 Package                        Architecture        Version                          Repository                          Size
================================================================================================================================
Installing:
 docker                         x86_64              25.0.14-1.amzn2023.0.2           amazonlinux                         46 M
Installing dependencies:
 container-selinux              noarch              4:2.245.0-1.amzn2023             amazonlinux                         58 k
 containerd                     x86_64              2.2.1-1.amzn2023.0.1             amazonlinux                         24 M
 iptables-libs                  x86_64              1.8.8-3.amzn2023.0.2             amazonlinux                        401 k
 iptables-nft                   x86_64              1.8.8-3.amzn2023.0.2             amazonlinux                        183 k
 libcgroup                      x86_64              3.0-1.amzn2023.0.1               amazonlinux                         75 k
 libnetfilter_conntrack         x86_64              1.0.8-2.amzn2023.0.2             amazonlinux                         58 k
 libnfnetlink                   x86_64              1.0.1-19.amzn2023.0.2            amazonlinux                         30 k
 libnftnl                       x86_64              1.2.2-2.amzn2023.0.2             amazonlinux                         84 k
 pigz                           x86_64              2.5-1.amzn2023.0.3               amazonlinux                         83 k
 runc                           x86_64              1.3.4-1.amzn2023.0.2             amazonlinux                        3.9 M

Transaction Summary
================================================================================================================================
Install  11 Packages

Total download size: 75 M
Installed size: 282 M
Downloading Packages:
```

sudo service docker start

```
[ec2-user@ip-172-31-20-31 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-20-31 ~]$
```

sudo usermod -aG docker ec2-user

```
[ec2-user@ip-172-31-20-31 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-172-31-20-31 ~]$
```

Logout and reconnect

docker ps

```
[ec2-user@ip-172-31-20-31 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-20-31 ~]$
```

Deadline : 24/3/2026

Step 4: Install Docker Compose

sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" \
-o /usr/local/bin/docker-compose

```
[ec2-user@ip-172-31-20-31 ~]$ sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" \
-o /usr/local/bin/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0     0     0  --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0     0     0  --:--:-- --:--:-- --:--:--     0
100 30804k 100 30804k    0     0 88460k     0  --:--:-- --:--:-- --:--:-- 115.9M
[ec2-user@ip-172-31-20-31 ~]$
```

sudo chmod +x /usr/local/bin/docker-compose

```
[ec2-user@ip-172-31-20-31 ~]$ sudo chmod +x /usr/local/bin/docker-compose
[ec2-user@ip-172-31-20-31 ~]$
```

Verify:

docker-compose --version

```
[ec2-user@ip-172-31-20-31 ~]$ sudo chmod +x /usr/local/bin/docker-compose
[ec2-user@ip-172-31-20-31 ~]$ docker-compose --version
Docker Compose version v5.1.0
```

- **PHASE 2 — CREATE WEB APP (WITH DOCKER + COMPOSE)**

Step 5: Create Project

mkdir webapp
cd webapp

```
[ec2-user@ip-172-31-20-31 ~]$ mkdir webapp
cd webapp
[ec2-user@ip-172-31-20-31 webapp]$
```

Step 6: Create HTML

nano index.html

```
[ec2-user@ip-172-31-20-31 webapp]$ nano index.html
[ec2-user@ip-172-31-20-31 webapp]$
```

```
<!DOCTYPE html>
<html>
<body>
<h1>Hello from AWS Docker CI/CD</h1>
<p>Version 1</p>
</body>
</html>
```

Step 7: Create Dockerfile

nano Dockerfile

```
[ec2-user@ip-172-31-20-31 webapp]$ nano Dockerfile
[ec2-user@ip-172-31-20-31 webapp]$
```

```
FROM nginx
ARG CACHE_BUST
COPY index.html /usr/share/nginx/html/index.html
```

Step 8: Create docker-compose.yml

nano docker-compose.yml

```
[ec2-user@ip-172-31-20-31 webapp]$ nano docker-compose.yml
[ec2-user@ip-172-31-20-31 webapp]$
```

```
services:
  app:
    image: <your-docker-username>/webapp:latest
    container_name: webapp
    ports:
      - "8080:80"

  watchtower:
    image: containrrr/watchtower
    container_name: watchtower
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    command: --interval 30 --cleanup
```

Step 9: (Optional) Test Locally

docker build -t webapp .

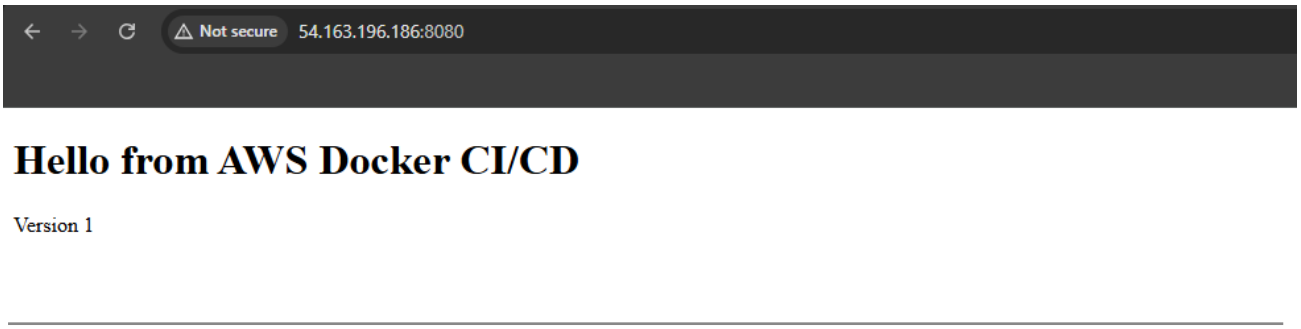
```
[ec2-user@ip-172-31-20-31 webapp]$ docker build -t webapp .
[+] Building 5.1s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 171B
 => [internal] load metadata for docker.io/library/nginx:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 196B
 => [1/2] FROM docker.io/library/nginx:latest@sha256:dec7a90bd0973b076832dc56933fe876bc014929e14b4ec49923951405370112
 => => resolve docker.io/library/nginx:latest@sha256:dec7a90bd0973b076832dc56933fe876bc014929e14b4ec49923951405370112
 => => sha256:dead9cf6bae3a0447c3148dd1afb374a3c8339b8875298bd3130387f64abf879 2.29kB / 2.29kB
 => => sha256:1a1e6313642049664ed59099f3ca37a672486049af68395c62dd885a9bf4de35 9.09kB / 9.09kB
 => => sha256:ec781dee3f4719c2ca0dd9e73cb1d4ed834ed1a406495eb6e44b6dfaad5d1f8f 29.78MB / 29.78MB
```

docker run -d -p 8080:80 webapp

```
[ec2-user@ip-172-31-20-31 webapp]$ docker run -d -p 8080:80 webapp
9c768037a3c078853e83249284b2c3362648bdd69f54f09422080b6d66d0fcfe
[ec2-user@ip-172-31-20-31 webapp]$
```

Open:

http://<EC2-PUBLIC-IP>:8080



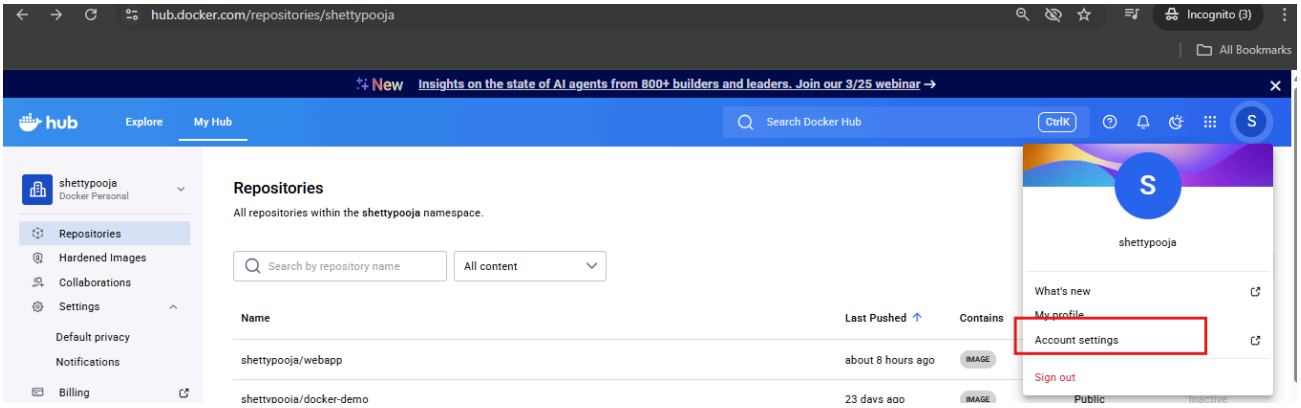
- **PHASE 3 — DOCKER HUB**

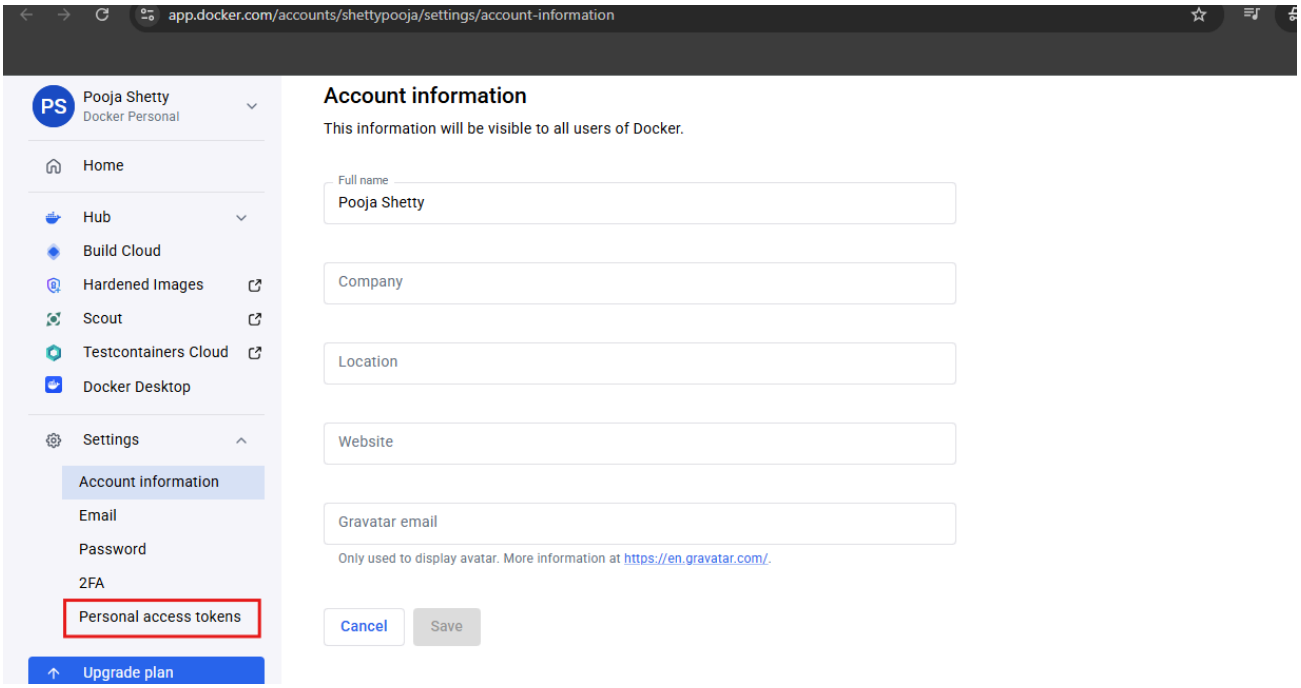
Step 10: Login

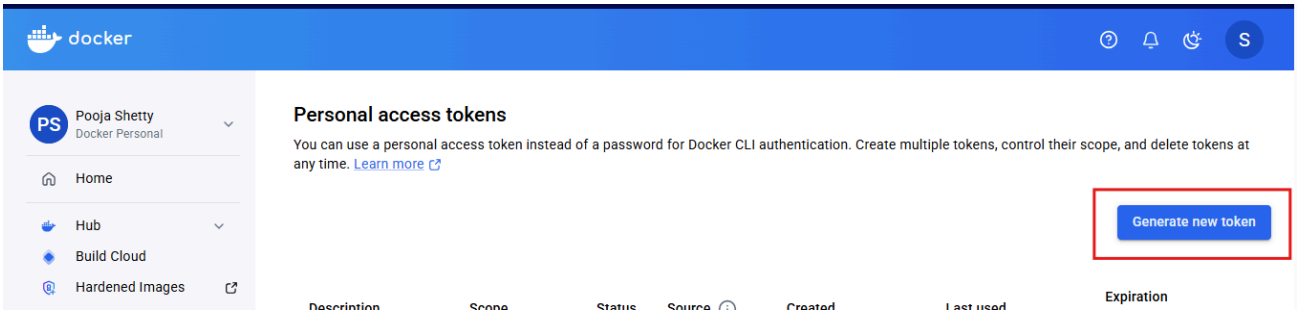
docker login

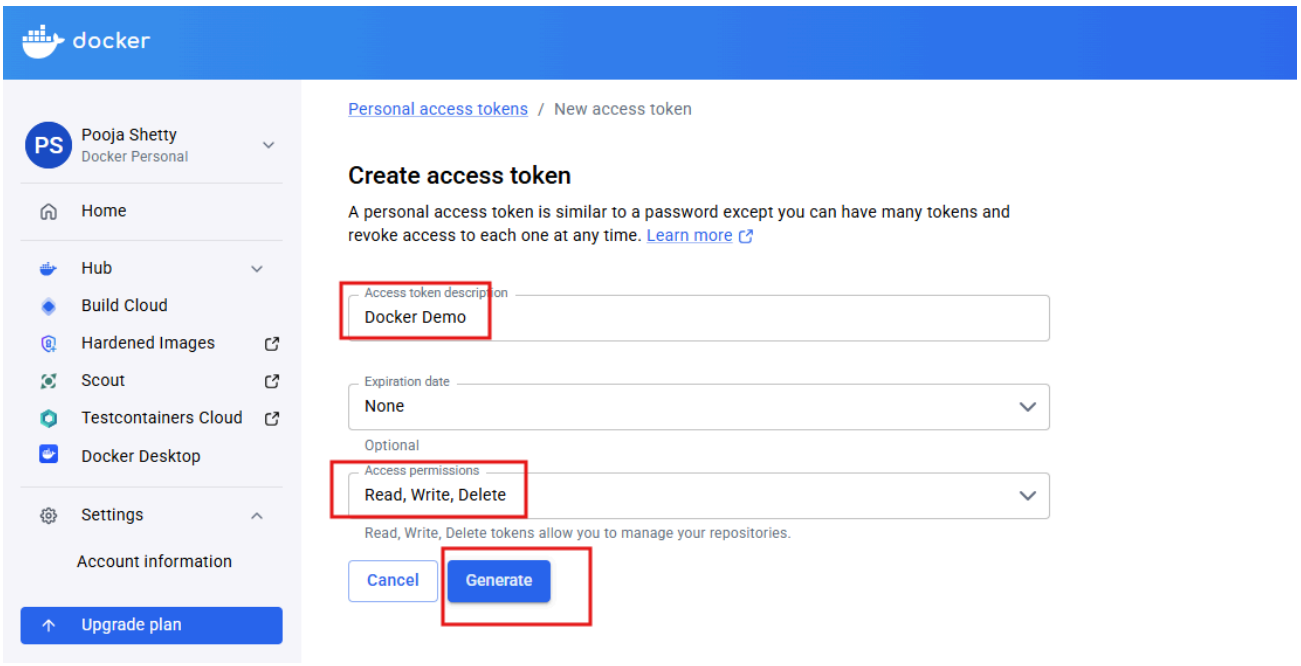
```
[ec2-user@ip-172-31-20-31 webapp]$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create a
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Le
ocs.docker.com/go/access-tokens/

Username: shettypooja
Password:
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-172-31-20-31 webapp]$
```

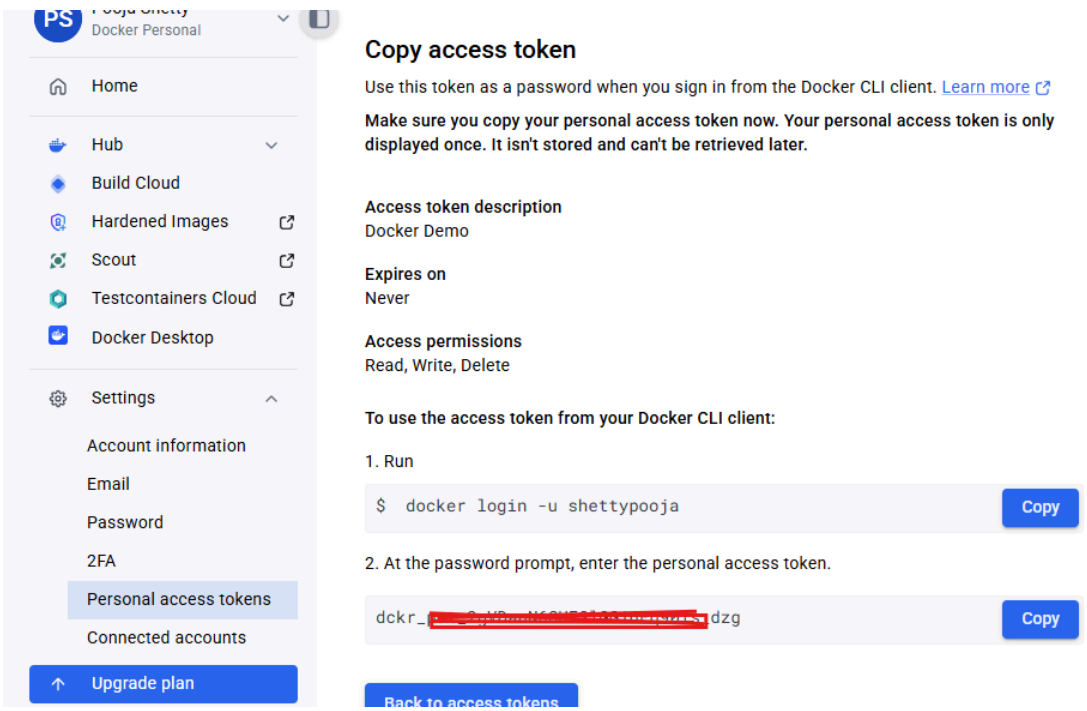
In place of password with generate the Personal Access Token (PAT) or generate password (if using gmail login)


hub.docker.com/repositories/shettypooja
Incognito (3)
All Bookmarks
New
Insights on the state of AI agents from 800+ builders and leaders. Join our 3/25 webinar →
hub
Explore
My Hub
Search Docker Hub
Ctrl K
shettypooja
Docker Personal
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Repositories
All repositories within the shettypooja namespace.
Search by repository name
All content
Name
Last Pushed
Contains
shettypooja/webapp
about 8 hours ago
IMAGE
shettypooja/docker-demo
23 days ago
IMAGE
Public
Inactive
S
shettypooja
What's new
My profile
Account settings
Sign out


app.docker.com/accounts/shettypooja/settings/account-information
PS
Pooja Shetty
Docker Personal
Home
Hub
Build Cloud
Hardened Images
Scout
Testcontainers Cloud
Docker Desktop
Settings
Account information
Email
Password
2FA
Personal access tokens
Upgrade plan
Account information
This information will be visible to all users of Docker.
Full name
Pooja Shetty
Company
Location
Website
Gravatar email
Only used to display avatar. More information at https://en.gravatar.com/.
Cancel
Save


docker
S
PS
Pooja Shetty
Docker Personal
Home
Hub
Build Cloud
Hardened Images
Personal access tokens
You can use a personal access token instead of a password for Docker CLI authentication. Create multiple tokens, control their scope, and delete tokens at any time. Learn more
Generate new token
Description
Scope
Status
Source
Created
Last used
Expiration





Put the Personal Access Token (PTA) as password.
Note password is not shown while typing

Step 11: Tag Image

docker tag webapp <your-docker-username>/webapp:latest

```
[ec2-user@ip-172-31-20-31 webapp]$ docker tag webapp shettypooja/webapp:latest
[ec2-user@ip-172-31-20-31 webapp]$
```

Step 12: Push Image

docker push <your-docker-username>/webapp:latest

```
[ec2-user@ip-172-31-20-31 webapp]$ docker push shettypooja/webapp:latest
The push refers to repository [docker.io/shettypooja/webapp]
753f6bfe6dca: Pushed
f60f9b3b726f: Pushed
e858199e645c: Pushed
1d28fb599436: Pushed
8a8acef9ebbb: Pushed
de70228576e2: Pushed
3de6db9e4b16: Pushed
188c9b34dfbe: Pushed
latest: digest: sha256:a4adae507c6bae8b177d50b6bb2ed93a9cb213c0d05f9eea2628478c1329f604 size: 1985
[ec2-user@ip-172-31-20-31 webapp]$
```

🔹 PHASE 4 — GITHUB (CI)

Step 13: Create Repository

Upload:

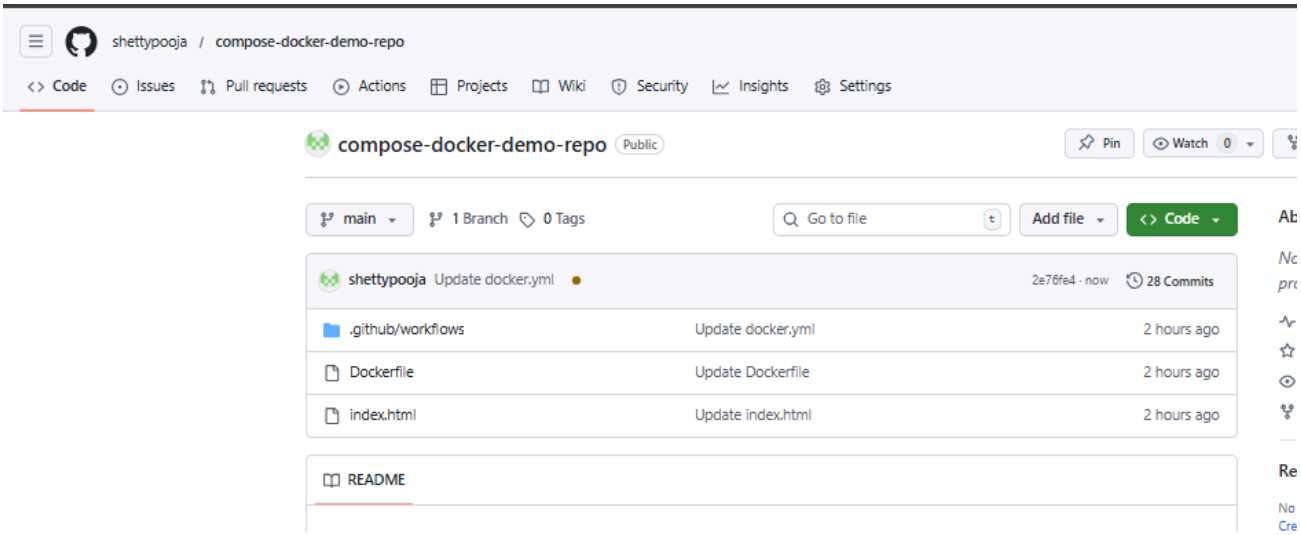
- index.html
- Dockerfile

Step 14: Add Workflow (into the repo)

docker.yml

```
name: Build and Push Docker Image

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      # 1. Checkout code
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Login to Docker Hub
```

```
- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

# 3. Build Docker image (force new image every time)
- name: Build Docker image
  run: |
    docker build \
      --build-arg CACHE_BUST=$(date +%s) \
      -t ${{ secrets.DOCKER_USERNAME }}/webapp:latest .

# 4. Push Docker image
- name: Push Docker image
  run: |
    docker push ${{ secrets.DOCKER_USERNAME }}/webapp:latest
```

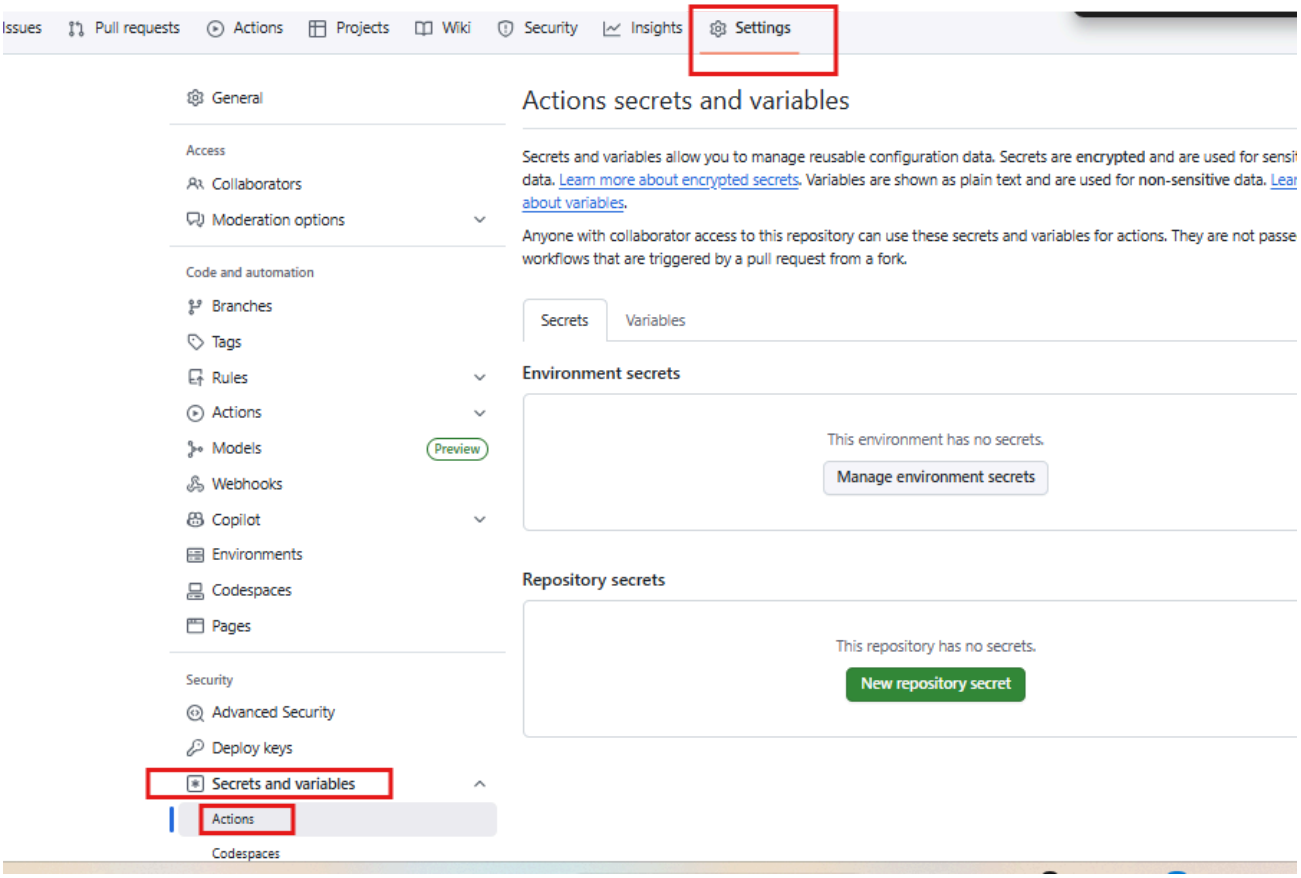


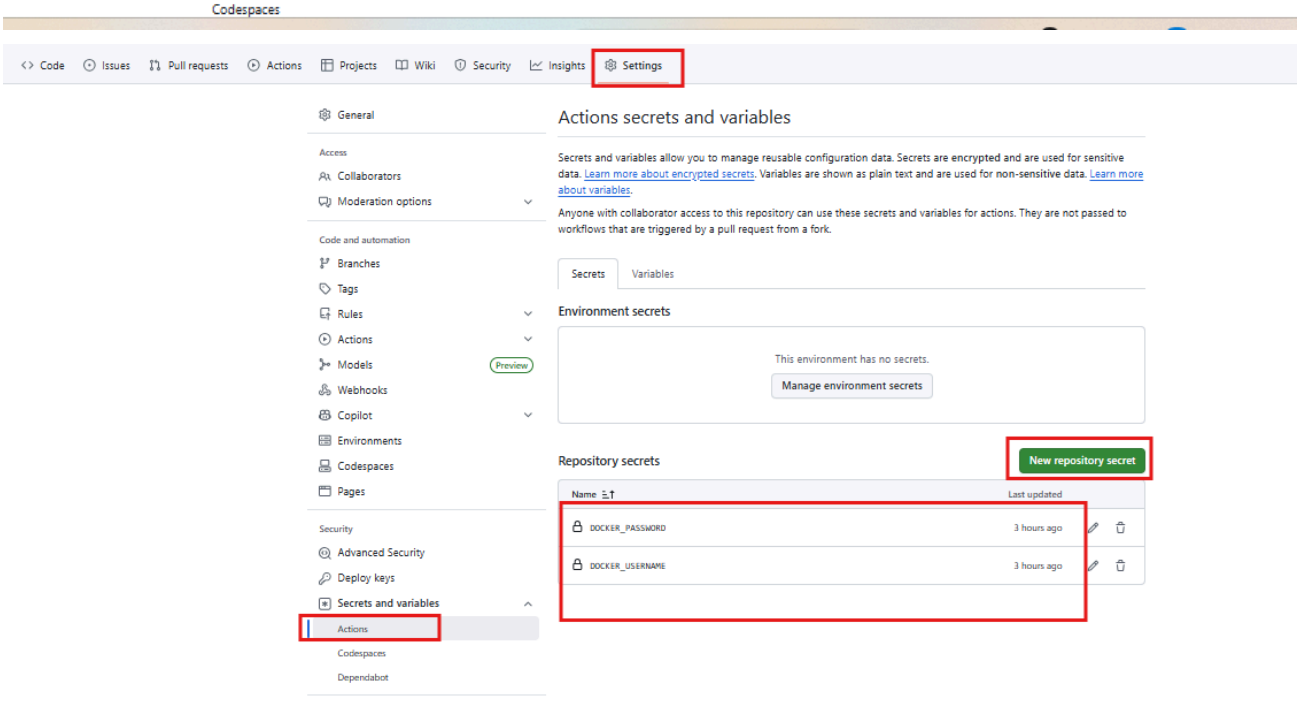
https://github.com/shettypooja/compose-docker-demo-repo.git

Deadline: 26/3/2026

Step 15: Add Secrets

- DOCKER_USERNAME
- DOCKER_PASSWORD





🔹 PHASE 5 — DEPLOY USING DOCKER COMPOSE

Step 16: Start Application

docker-compose up -d

```
[ec2-user@ip-172-31-20-31 webapp]$ docker-compose up -d
[+] up 7/7
 ✓ Image containrrr/watchtower Pulled
 ✓ Network webapp_default      Created
 ✓ Container webapp            Started
 ✓ Container watchtower        Started
[ec2-user@ip-172-31-20-31 webapp]$
```

Step 17: Verify

docker ps

```
[ec2-user@ip-172-31-20-31 webapp]$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED              STATUS                     PORTS                                      NAMES
49df1f090a50   shettypooja/webapp:latest   "/docker-entrypoint.…"   About a minute ago   Up About a minute          0.0.0.0:80->80/tcp, :::80->80/tcp          webapp
e3fe7400e44b   containrrr/watchtower    "/watchtower --inter…"   About a minute ago   Up About a minute (healthy)   8080/tcp                                   watchtower
9c768037a3c0   webapp                   "/docker-entrypoint.…"   33 minutes ago       Up 33 minutes              0.0.0.0:8080->80/tcp, :::8080->80/tcp      elegant_ganguly
[ec2-user@ip-172-31-20-31 webapp]$
```

🔹 PHASE 6 — TEST CI/CD

Step 18: Modify Code

Update `index.html` in GitHub:

<h1>Version 2</h1>

Step 19: Automatic Flow

GitHub (Code Push)
↓
GitHub Actions (Build Docker Image)
↓
Push Image to Docker Hub
↓
Watchtower (Running in EC2 checks Docker Hub)
↓
Pull Latest Image
↓
Auto Update Container on EC2
↓
Updated Web App Available (Browser)

Step 20: Verify Output

Open: (since the port is 8080, just the IP address is ok}

http://<EC2-PUBLIC-IP>

TROUBLESHOOTING

Check Watchtower

docker logs watchtower

Force Reset

docker rm -f $(docker ps -aq)
docker-compose up -d

FINAL RESULT

Edit code → git push → auto deploy in ~30 sec